Developer guide

Welcome. This guide gets a new engineer from a fresh clone to a merged change. Read it with `../ARCHITECTURE.md` (docs/ARCHITECTURE.md) (module boundaries and data flow) and `DATA-MODEL.md` (docs/developer/DATA-MODEL.md).

1. The product in one paragraph

RoadSaathi is a towing and roadside-assistance marketplace. **Customers** book on a bilingual website, **drivers and fleet companies** (partners) receive and fulfil jobs in a driver app and partner portal, and **staff** run operations from an admin back-office. One Next.js deployable serves all four surfaces, backed by one Postgres database.

2. Stack

Layer	Choice	Notes
Framework	**Next.js 16** (App Router, React 19, Server Components, Server Actions)	⚠ This Next.js has breaking changes vs. older versions. Read `node_modules/next/dist/docs/` before using an unfamiliar API (see `AGENTS.md`)
Language	TypeScript (strict)	`npm run lint` = `tsc --noEmit`
Database	PostgreSQL via **Prisma 7** (`@prisma/adapter-pg`)	Client generated to `src/generated/prisma` (git-ignored)
Styling	Tailwind CSS 4	Safety-yellow and charcoal design tokens; brand colours come from settings
Validation	Zod 4	Every action input and every settings group
Auth	Own OTP + DB sessions (`jose` only for small signed tokens)	Firebase phone auth optional
Maps	Leaflet + OpenStreetMap (Nominatim, OSRM); Google optional	Provider interface in `modules/geo/providers.ts`
Email / SMS	Nodemailer (SMTP), MSG91 or mock	Everything logged in `NotificationLog`
Payments	Razorpay (feature-flagged)	Webhook at `/api/webhooks/razorpay`
Tests	Vitest	`npm test`
Hosting	Vercel + Neon (primary), Render/Docker (secondary)	`docs/DEPLOY.md`

3. Getting started

```
git clone <repo> && cd TowingService
cp .env.example .env
npm install                       # also runs prisma generate
docker compose up -d db mailpit   # Postgres on 5432, Mailpit UI on http://localhost:8025
npx prisma migrate deploy
SEED_DEMO=true npm run db:seed    # catalog, settings, legal pages + demo partners
EXPOSE_DEV_OTP=true npm run dev   # http://localhost:3000 ; OTP codes shown on screen in dev
```

Sign in at `/en/login` with `ADMIN_EMAIL` (default `admin@roadsaathi.example`) and open `/en/admin`. Demo partner logins are in the README. To try the full loop: book as a customer → sign in as a demo driver in another browser → *Go online* → accept → advance → enter PIN → complete.

Command	What it does
`npm run dev`	Dev server with hot reload
`npm run lint`	Type-check the whole project
`npm test`	Unit tests (Vitest)
`npm run build`	`prisma generate` + production build
`npm run db:migrate`	Create a new migration from `schema.prisma` changes (dev only)
`npm run db:deploy`	Apply migrations (CI / production)
`npm run db:seed`	Idempotent seed; never overwrites admin edits

4. Repository map

```
src/app/[locale]/(site)/      public website + booking, tracking, invoice, legal, account
src/app/[locale]/driver/      driver app (PWA)
src/app/[locale]/partner-portal/  fleet-company portal
src/app/[locale]/admin/       back-office
src/app/api/                  route handlers: health, geo, track, driver state/location, event
beacon, cron, Razorpay webhook
src/modules/<name>/           business logic, one folder per bounded context
    service.ts                the module's public functions (other modules call only these)
    actions.ts                "use server" actions called by UI; thin: auth → rate-limit → service →
track
    *.test.ts                 unit tests next to the code
src/shared/lib/               env, db, guard (rate limit, audit), utils, markdown, localized text
src/shared/ui | layout | client   design system, shells, client components
src/i18n/                     en.ts, hi.ts dictionaries (hi is type-checked against en)
src/proxy.ts                  locale routing (Next 16 "proxy", formerly middleware)
prisma/                       schema, migrations, seed + seed-data (catalog, rate cards, FAQs, legal
pages, contracts)
docs/                         this documentation set
marketing/                    demo-film pipeline (separate package)
```

5. Conventions (please follow)

1. **Module boundaries.** Pages and actions call module services. A module never imports from `app/`. `shared/` never imports a module, except UI shells that need brand or auth. The `admin` module may *read* across modules but *writes* through their services.
2. **Server actions re-check auth.** Layout guards are not enough. Every action starts with `getCurrentUser()` or `assertRole([...])` (`modules/auth/guards.ts`) and returns an `ActionResult` (`{ ok: true, data } | { ok: false, error }`).
3. **Validate every input with Zod** at the module boundary, even when the UI already validates.
4. **Rate-limit public and OTP endpoints** with `rateLimit(key, limit, windowSec)` from `shared/lib/guard.ts`.
5. **Money is integer paise.** Never use floats for rupees. Format only at the edge with `formatINR`.
6. **Every booking status change goes through** `canTransition()` and writes a `BookingEvent`. Never update `Booking.status` directly.
7. **Concurrency:** use a conditional `updateMany` (check the affected count) for races such as two drivers accepting the same offer. See `acceptOffer` in `modules/dispatch/service.ts`.
8. **All customer-visible text exists in English and Hindi.** UI strings go in `src/i18n`; DB content uses the `Localized` `{ en, hi }` JSON shape and is read with `L(value, locale)`.
9. **Brand is a variable.** Never hard-code "RoadSaathi". Use `getBrand()` and `{{brand}}` placeholders in content.
10. **Settings over constants.** Business-tunable values belong in a settings group (`modules/settings/schemas.ts`) with a default, so ops can change them without a deploy.
11. **Caching:** readers use `unstable_cache` with tags (`settings`, `catalog`, `content`, `areas`). Writers call `updateTag(tag)` after a change.

12. **Side effects after the response:** notifications use `defer()` (built on `after()`) so a slow SMTP server never slows a booking.
13. **Analytics:** add new event names to `EVENTS` in `modules/analytics/track.ts` *and* `docs/tracking-plan.md`. Never put PII in event props.
14. **Audit:** staff actions that change data call `audit({ actorId, action, entity, entityId, meta })`.
15. **Comments:** explain *why*, briefly. Match the surrounding density.

6. How-to recipes

Add a setting

1. Add the field with a default to the group's Zod schema in `src/modules/settings/schemas.ts` (`schema.parse({})` must still produce a working configuration).
2. Add the field spec to `src/modules/settings/fields.ts` so it appears in Admin → Settings.
3. Read it with `getSetting("group")` or `getSettings()`. No migration is needed (settings are JSON rows).

Add an admin-editable table (catalog or content)

1. Add the Prisma model and run `npm run db:migrate -- --name <change>`.
2. Register it in `src/modules/admin/resources.ts` (fields, columns, cache tags, optional `prepare`). The generic `admin/manage/[resource]` pages give list, create, edit and delete for free.

Add a booking status or transition

1. Update the enum in `schema.prisma` and migrate.
2. Update `NEXT`, `STATUS_LABEL` and, if drivers trigger it, `DRIVER_NEXT` in `modules/bookings/state.ts`.
3. Extend `state.test.ts`. Check notifications (`modules/notifications/events.ts`) and the tracking UI.

Add a notification

Add a function in `modules/notifications/events.ts` that builds the email with `renderEmail` and sends through `sendEmail` / `sendSms`. Respect the toggles in the `notifications` settings group. Call it with `defer()`.

Add a legal page or contract version

Legal pages: Admin → Content → Legal pages, or add to `pages` in `prisma/seed-data.ts` for new installs. The seed **does not overwrite** existing rows, so update production content through the admin. Contract templates: create a new **version**. Signed contracts keep their immutable `renderedBody`.

Swap the map provider

Implement the `GeoProvider` interface in `modules/geo/providers.ts` and select it in Admin → Settings → Maps (or `MAP_PROVIDER`).

Add an AI feature

Follow `docs/business/05-ai-roadmap.md` §5.3: put the provider behind an interface (like geo), call existing module services rather than writing to tables directly, keep humans in the loop for money and suspension decisions, and audit every automated decision.

7. Git workflow

- Branch from `main`: `feat/<short-name>`, `fix/<short-name>`, `docs/<short-name>`.
- Small pull requests with a clear description. Before pushing: `npm run lint && npm test`.
- Conventional commit prefixes, matching history: `feat(scope):`, `fix(scope):`, `chore(scope):`, `docs(scope):`.
- Migrations are committed with the schema change. Never edit an applied migration; add a new one.
- `main` auto-deploys to Vercel and Render, so merge only green, reviewed changes. See `CONTRIBUTING.md` (docs/developer/CONTRIBUTING.md).

8. Where to look when something breaks

Symptom	First place to look
OTP not received	Admin → Notification log; `SMS_PROVIDER` / SMTP env; Firebase config
Booking stuck in "Finding driver"	Admin → Dispatch board; driver `lastSeenAt` freshness (Settings → Dispatch); cron `/api/cron/tick`
Wrong fare	`modules/pricing/engine.ts` (+ tests), rate card row, pricing settings (night hours, surge)
Settings change not visible	Cache tag not updated, or a 120-second `revalidate` window
Webhook not processed	`RAZORPAY_WEBHOOK_SECRET`; signature check in `modules/payments/service.ts`
Health check failing	`GET /api/health` (DB connectivity)

9. Known technical debt (prioritised)

1. No CI, error monitoring or log retention (PRD L7, L11).
2. Tests cover the pricing engine, state machine and utils only. Add integration tests for booking → dispatch → completion, and for auth guards on every action.
3. Polling (5 s) for tracking and offers; move to SSE/WebSockets with Redis-backed locations at scale.
4. CSP allows `'unsafe-inline'` scripts (needed by inline GA bootstrap and Razorpay). Move to nonces.
5. Public OSM endpoints are fair-use only. Self-host or switch providers before marketing spend.
6. No file storage yet (needed for KYC documents and photos). Pick S3-compatible storage with signed URLs.